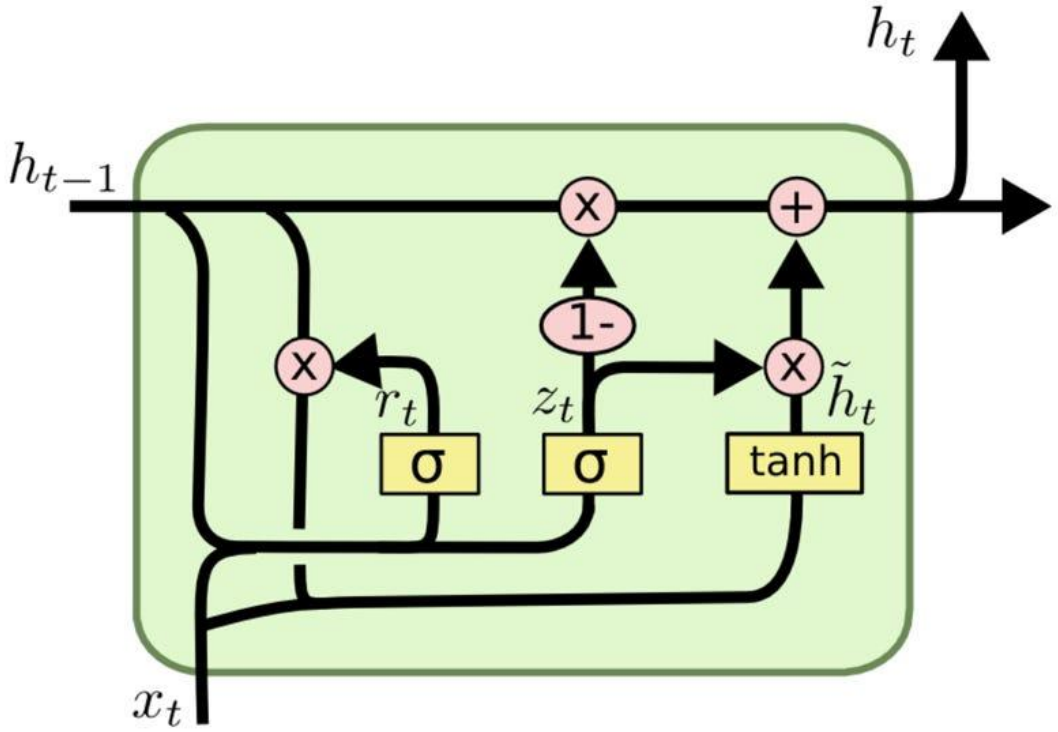
방법론 세미나

이지훈

목차

* GRU
* RNN의 기울기 소실 문제
* LSTM과 GRU 비교
* GRU 실행과정
* 실습
* 결론

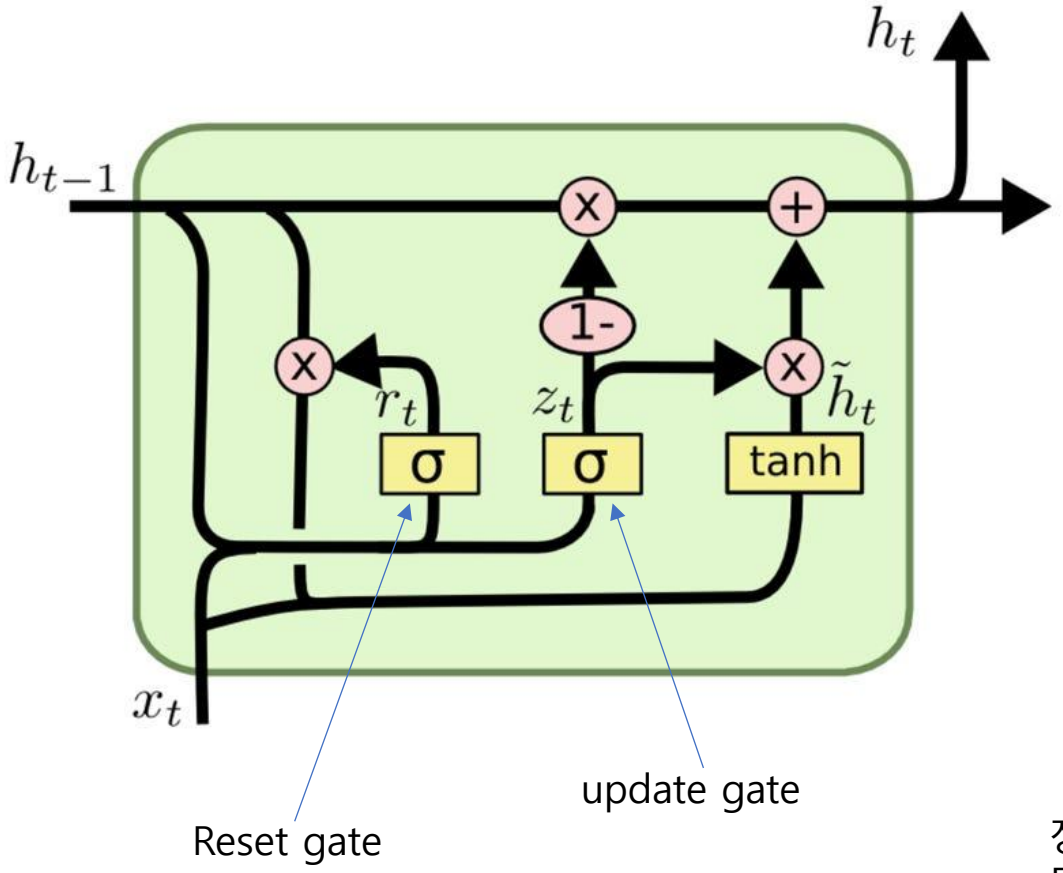
GRU



GRU

- GRU (Gated Recurrent Unit)
- GRU는 RNN의 일종으로 LSTM을 개선하여 파라미터 개수를 줄인 순환 신경망의 게이트 구조
- 2014년 뉴욕 대학교의 조경현 교수 외 6인 발표
- LSTM의 장기기억능력은 보존하면서 연산은 줄임

GRU

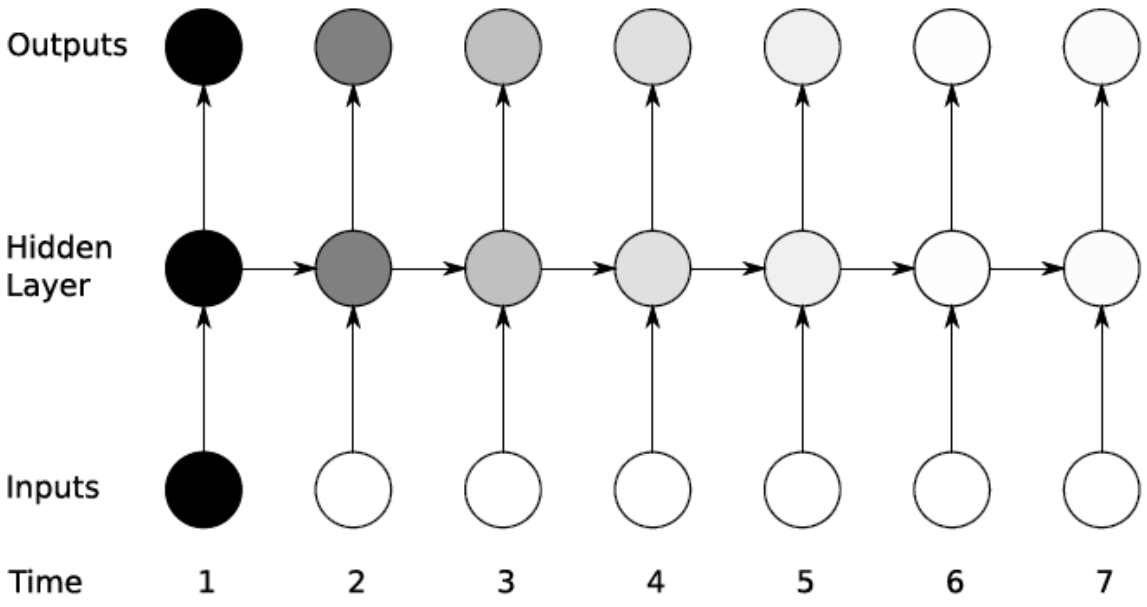


GRU

- Update gate와 reset gate를 사용하여 기존RNN의 장기의존성 문제를 해결하는 신경망 구조.
- 각 게이트는 정보를 선택적으로 업데이트 또는 초기화 하도록 설계되어, 긴 시퀀스의 데이터 학습 때 기울기 소실 문제를 완화.

장기의존성 문제: 기울기 소실문제로 인해 네트워크가 과거의 정보를 제대로 학습하지 못하고 이로 인해 장기적인 정보를 유지하는 것이 어려움

RNN

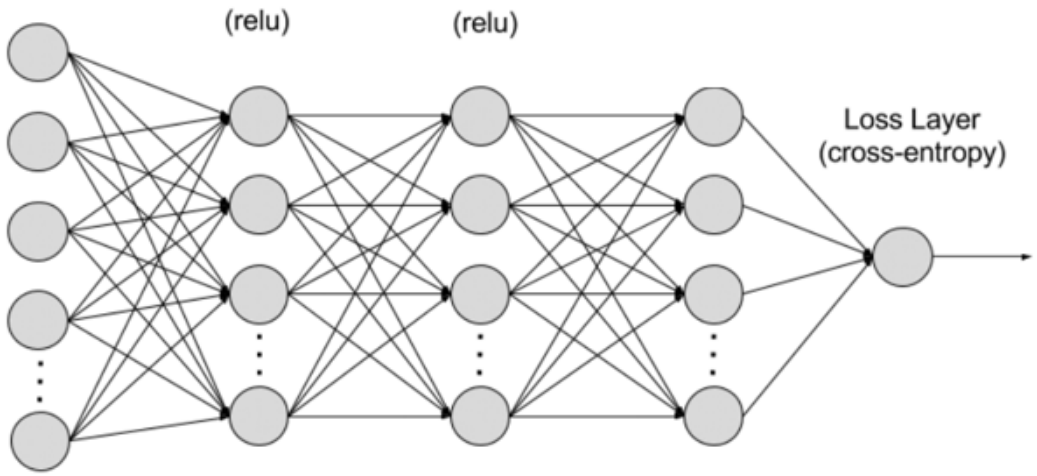


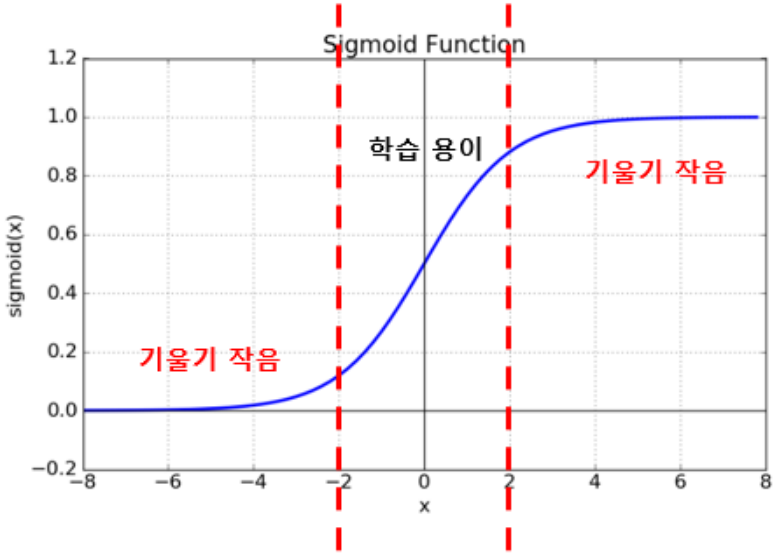
기울기 소실 문제

- RNN에서 결과의 오류를 바로잡기 위해 역전파를 하고, 이를 위해 각 layer들의 가중치를 조절한다, cost함수의 최적화를 위해 기울기를 구하게 되고 이때 기울기는 해당 함수의 편미분 값이다.

기울기 소실 문제

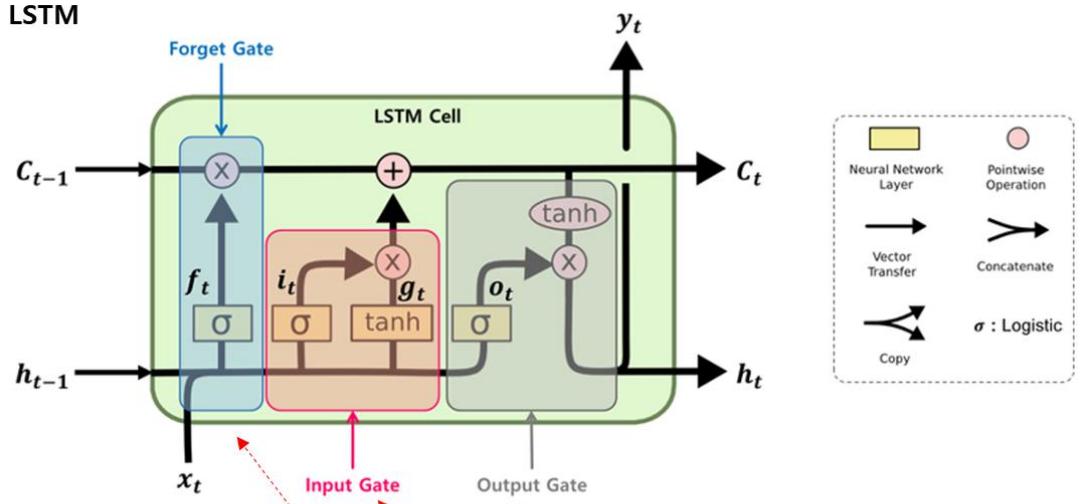
- 활성화 함수는 대부분의 영역에서 기울기가 0에 가깝고 오류 역전파 층은 거칠수록 활성화 함수의 편미분이 계속 곱해지는 구조인데, 곱해지는 값이 0또는 굉장히 작은 수일 확률이 높아 결과적으로 망이 깊어질수록(층이 많아질수록) 역전파 되는 오차의 값이 0에 수렴하여 가중치 갱신이 이루어지지 않게 된다(기울기가 소실).
- 장기의존성 문제: 기울기 소실 문제로 인해 과거의 정보를 제대로 학습하지 못하여 장기적인 정보를 유지하는 것이 어려움



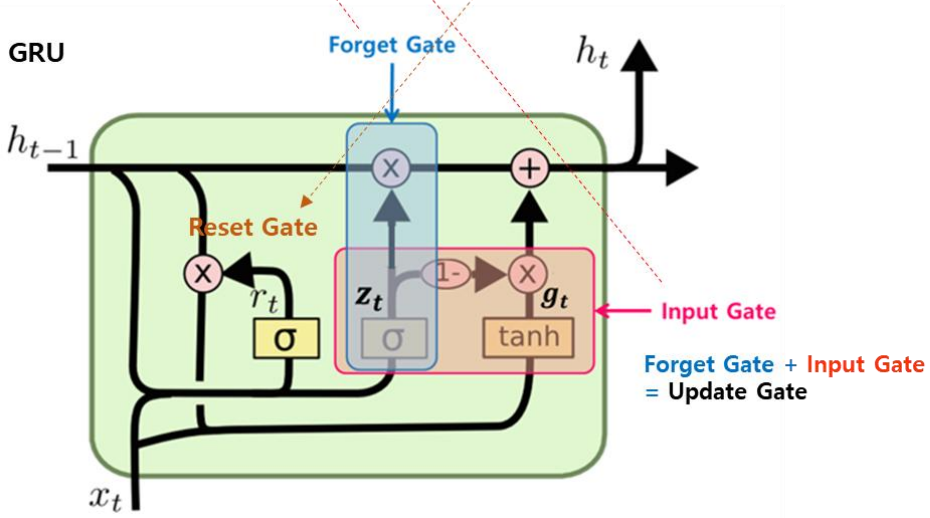


비교

LSTM



GRU



구분	LSTM	GRU
출시 연도	1997년	2014년
게이트 수	3개 (입력/망각/출력)	2개 (업데이트/리셋)
복잡성	상대적 복잡	상대적 단순
계산 비용	고가	저가
셀 상태/게이트 계산	여러 게이트를 통해 셀 상태와 숨겨진 상태를 모두 유지(계산)	리셋 게이트와 업데이트 게이트를 통해 숨겨진 상태만 유지(계산)
기억 용량	장기적 정보 유지에 더 강함, 긴 시퀀스에서 효율적	상대적으로 간단한 구조로 인해 짧은 시퀀스에 효율적
학습속도	상대적 느림	상대적 빠름
사례	언어 모델링, 음성인식, 번역	실시간 데이터 처리, 빠른 응답이 필요한 서비스

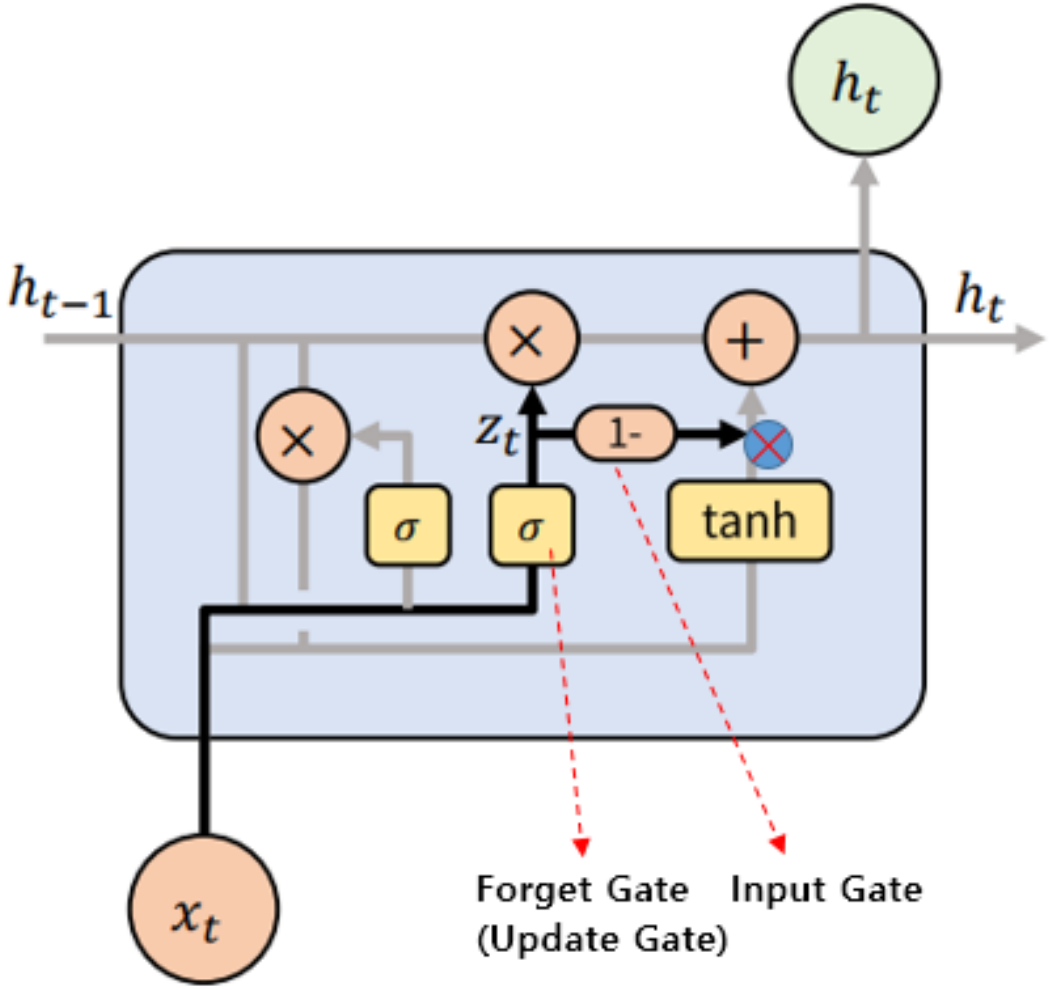
비교

```python
from tensorflow.python.keras.models import Sequential
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, LSTM, GRU, Input
from tensorflow.keras.optimizers import SGD

n_in = 1 # 입력층의 뉴런 수
n_mid = 20 # 중간층의 뉴런 수
n_out = 1 # 출력층의 뉴런 수

# 비교를 위한 LSTM 모델
input_lstm = Input(shape=(n_rnn, n_in))
lstm_layer = LSTM(n_mid, return_sequences=True)(input_lstm)
output_lstm = Dense(n_out, activation='linear')(lstm_layer)
model_lstm = Model(inputs=input_lstm, outputs=output_lstm)
model_lstm.compile(loss='mean_squared_error', optimizer=SGD()
print(model_lstm.summary())

# GRU
input_gru = Input(shape=(n_rnn, n_in))
gru_layer = GRU(n_mid, return_sequences=True)(input_gru)
output_gru = Dense(n_out, activation='linear')(gru_layer)
model_gru = Model(inputs=input_gru, outputs=output_gru)
model_gru.compile(loss='mean_squared_error', optimizer=SGD())
print(model_gru.summary())
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 10, 1)]           0

 lstm (LSTM)                 (None, 10, 20)            1760

 dense (Dense)               (None, 10, 1)             21

=================================================================
Total params: 1781 (6.96 KB)
Trainable params: 1781 (6.96 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
None
Model: "model_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 10, 1)]           0

 gru (GRU)                   (None, 10, 20)            1380

 dense_1 (Dense)             (None, 10, 1)             21

=================================================================
Total params: 1401 (5.47 KB)
Trainable params: 1401 (5.47 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

GRU가 파라미터가 더 적은 것을 알 수 있다.
즉 GRU의 학습시간이 더 짧다.

GRU

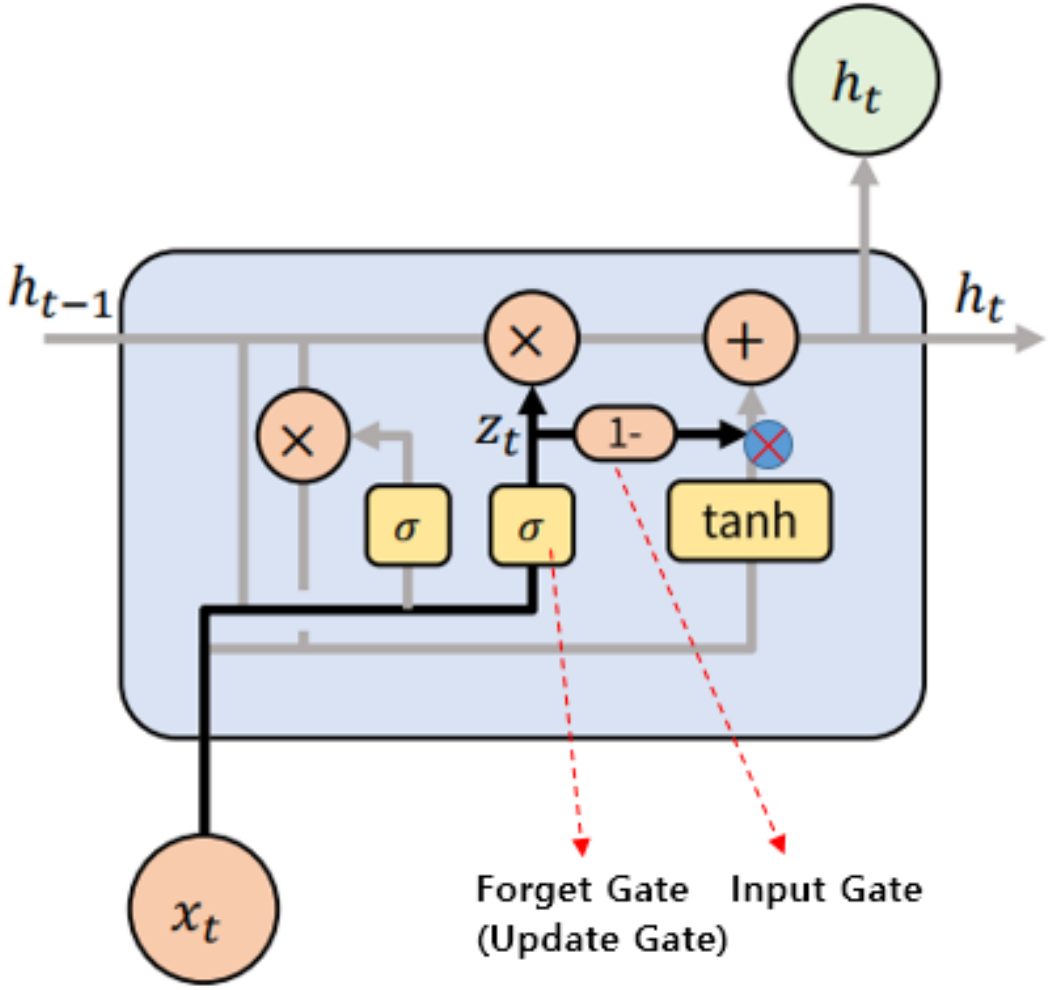


Update Gate

- LSTM의 Forget gate와 Input gate의 역할을 모두 담당.
- z_t가 이전 정보의 비율을 결정하고 1- z_t가 여기에 대응되는 현재 정보의 비율을 결정한다.
- 즉 과거의 정보를 얼마만큼 가져가고 현재 새로운 정보를 얼마만큼 가져갈지를 정해준다.

z_t는 Forget gate의 역할, 1- z_t는 Input gate의 역할

GRU

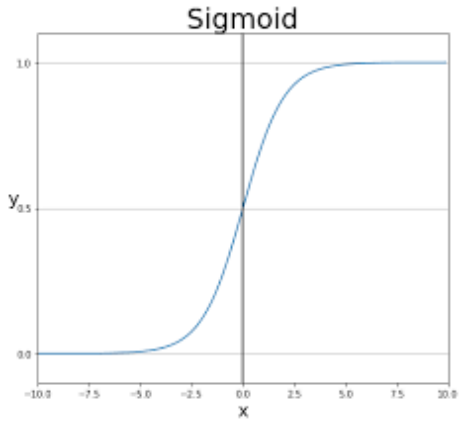


Update Gate

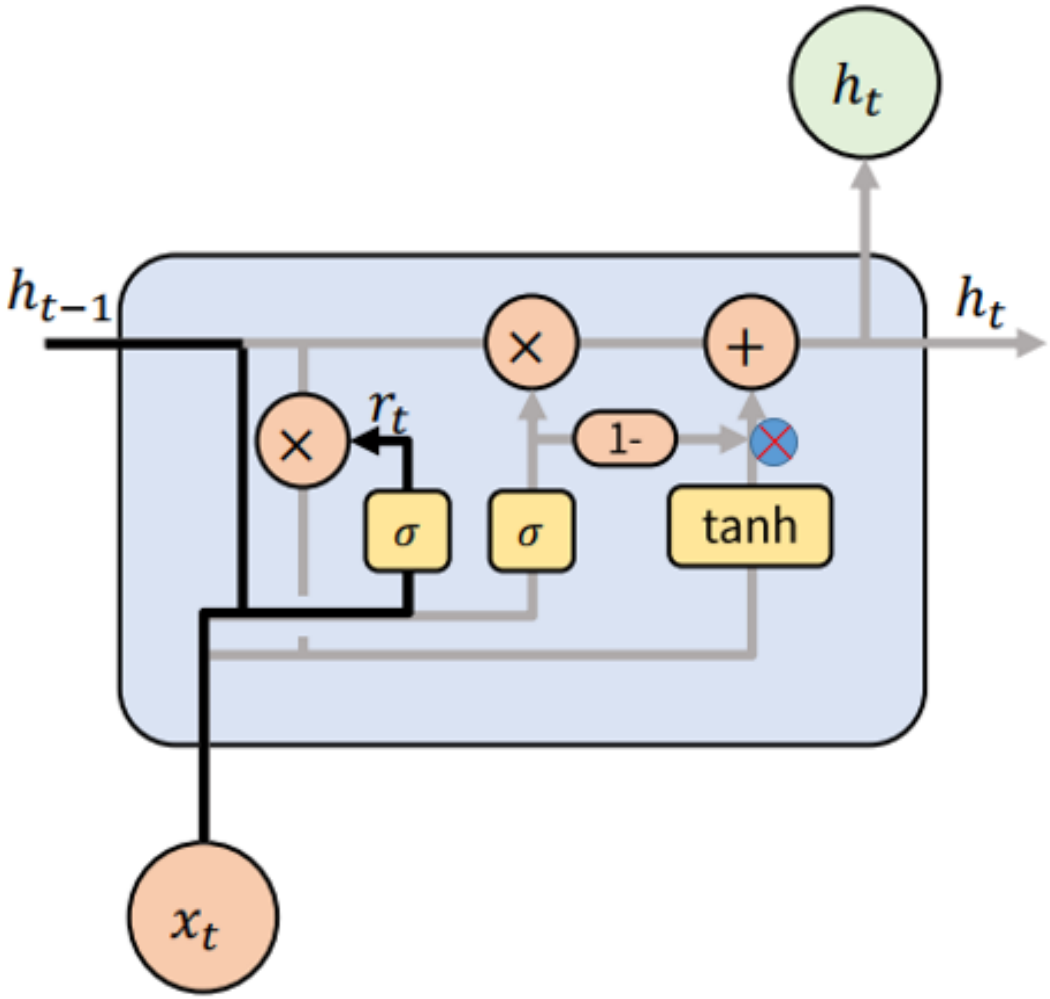
수식표현

Forget Gate : $z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$

Input Gate : $1 - z_t$



GRU



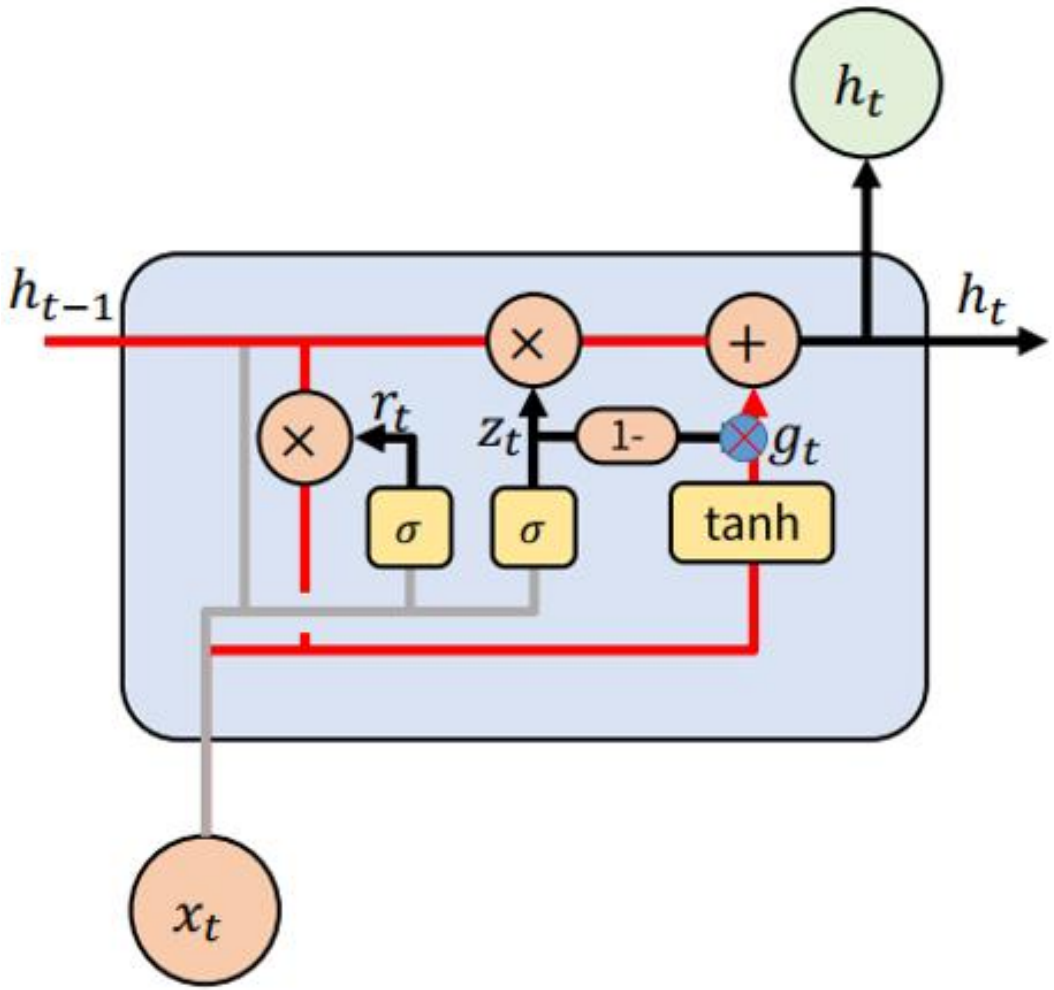
Reset Gate

- LSTM의 Output gate와 유사한 역할
- Reset Gate는 이전 정보에서 얼마만큼을 선택해서 내보낼지 결정.

수식표현

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$$

GRU



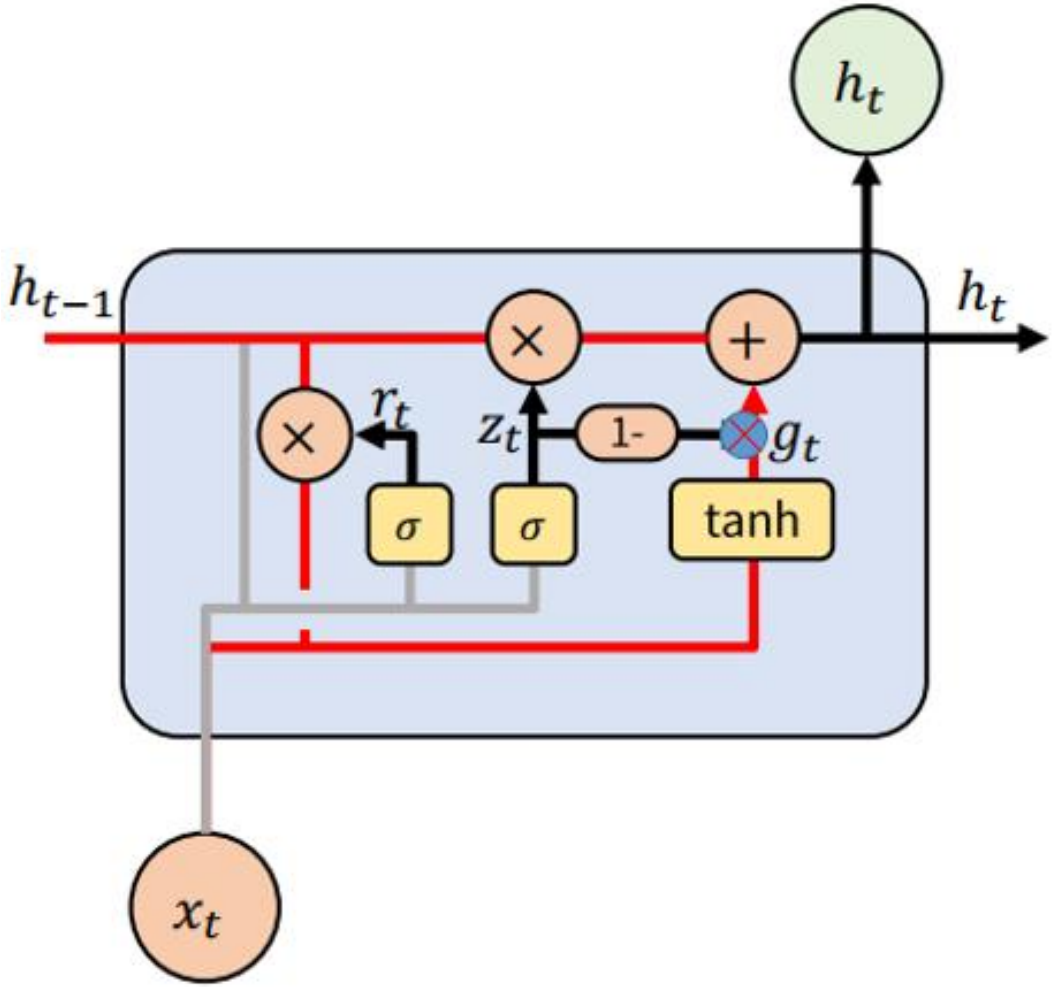
Hidden State

1) Reset gate, Update gate(Forget gate + Input gate)를 모두 적용하여 Hidden State를 계산한다

2) 이전 정보에 Reset gate의 결과를 곱하여 이전 정보 중 출력(h_t)으로 내보낼 정보를 선별한다

(이전 정보에서 어떤 정보를 내보내지 말아야 할지를 선별)

GRU



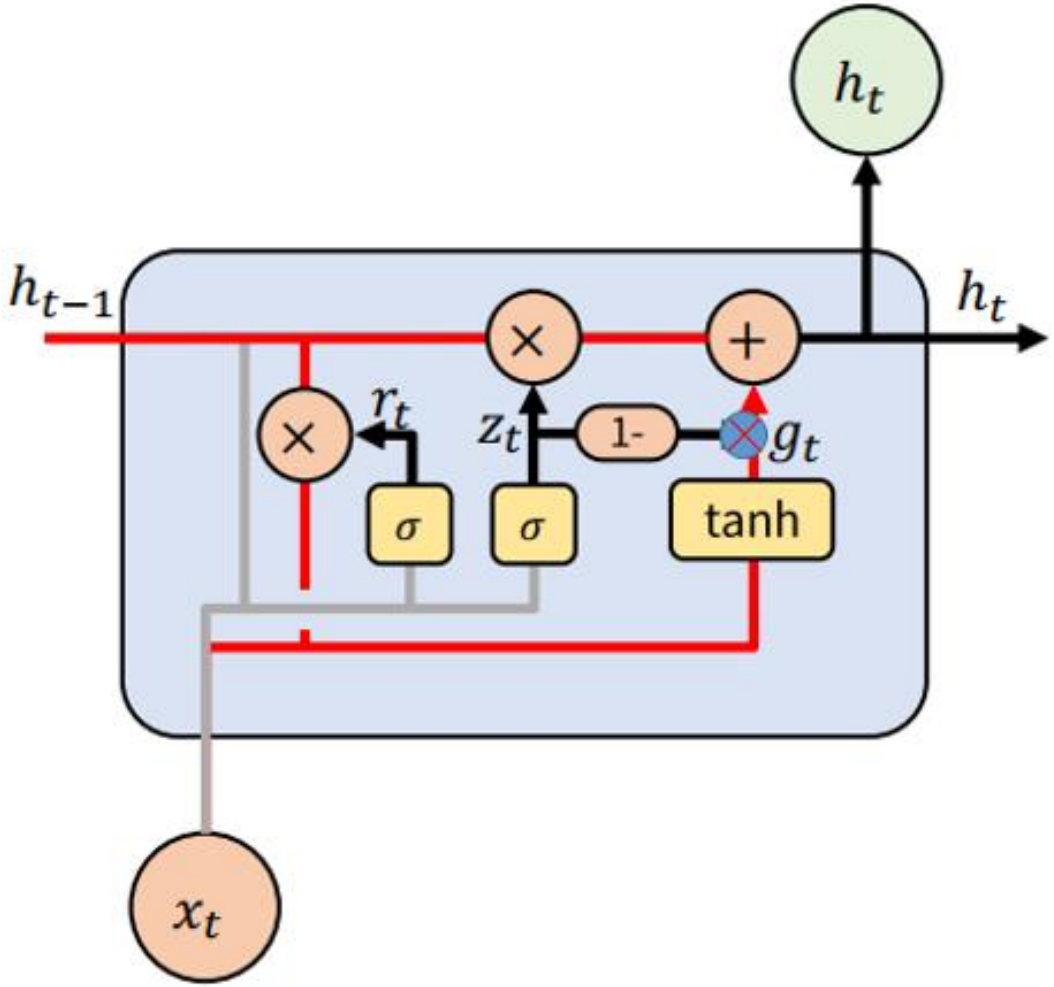
Hidden State

3) 선별한 결과에 tanh를 적용하여 -1~1 사이로 값을 바꾼다.

4) 다음 Update gate에서 구해진 현재 새로운 정보를 적용한 비율(1- z_t)를 than를 적용한 결과3) 에 곱하여 현재 정보 중 내보낼 정보를 계산한다.

5) Update gate에서 구해진 이전 정보의 비율(z_t)에 이전 정보 (h_t-1)를 곱하여 이전 정보 중 내보낼 값을 계산하고 이를 4) 에 더하여 최종적으로 optput로 내보낼 hidden state를 계산

GRU



Hidden State

수식표현

$$g_t = tanh(W_{xg}x_t + W_{hg}(r_t \odot h_{t-1}) + b_g)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot g_t$$

실습

```
# -- 훈련 데이터 생성 --
sin_x = np.linspace(-2*np.pi, 2*np.pi)
# # sin 곡선의 출력 범위는 -2π에서 2π까지

sin_y = np.sin(sin_x)  + 0.1*np.random.randn(len(sin_x))
# sin 함수에 난수 기반 노이즈 추가

n_sample = len(sin_x)-n_time  # 샘플 수
input_data = np.zeros((n_sample, n_time, n_in))  # 입력
correct_data = np.zeros((n_sample, n_out))  # 정답
for i in range(0, n_sample):
    input_data[i] = sin_y[i:i+n_time].reshape(-1, 1)
    correct_data[i] = sin_y[i+n_time:i+n_time+1]
```

https://colab.research.google.com/drive/1bGiwVU308h9rQTPTBKLg3MnyQr97f8Ve#scrollTo=bMfHfIpmV3F3

Sin곡선 예측

- 노이즈가 있는 sin곡선을 학습시키고 시계열 데이터를 예측

실습

```python
class GRULayer:
    def __init__(self, n_upper, n):
        # 파라미터의 초깃값
        self.w = np.random.randn(3, n_upper, n) / np.sqrt(n_upper)
          #slef w 입력x에 대한 가중치 3개 (업데이터게이트,리셋게이트,새로운기억)
        self.v = np.random.randn(3, n, n) / np.sqrt(n)
          #slef v 이전 은닉상태에 대한 가중치

    def forward(self, x, y_prev):
        a0 = sigmoid(np.dot(x, self.w[0]) + np.dot(y_prev, self.v[0]))
        #  업데이트 게이트
        a1 = sigmoid(np.dot(x, self.w[1]) + np.dot(y_prev, self.v[1]))
        # 리셋 게이트 (이전 은닉상태를 얼마나 잊을지 결정)
        a2 = np.tanh(np.dot(x, self.w[2]) + np.dot(a1*y_prev, self.v[2]))
        # 새로운 기억 (현재 입력과 이전 은닉 상태를 사용해 새로운 기억 생성)
        self.gates = np.stack((a0, a1, a2))

        self.y = (1-a0)*y_prev + a0*a2
        # 출력 (업데이트 게이트를 사용해 과거 정보를 얼마나 반영할지 결정)
```

GRU층

- 업데이트 게이트를 통해 새로운 정보와 기존 정보를 얼마나 반영할지 결정
- 리셋 게이트를 통해 이전 상태를 얼마나 반영할지 조절
- 새로운 기억을 통하여 현재 입력과 이전 상태를 바탕으로 새로운 정보를 생성 및 얼마나 반영할지 결정
- 업데이트 게이트를 사용해 기존 상태와 새로운 기억을 적절히 섞어 최종 은닉 상태를 만든다

실습

```
# 순전파 GRU층
y_rnn = np.zeros((len(x_mb), n_time+1, n_mid))
gates_rnn = np.zeros((3, len(x_mb), n_time, n_mid))
y_prev = y_rnn[:, 0, :]
for i in range(n_time):
    x = x_mb[:, i, :]
    gru_layer.forward(x, y_prev) #gru층을 통해 은닉 상태를 업데이트

    y = gru_layer.y
    y_rnn[:, i+1, :] = y
    y_prev = y

    gates = gru_layer.gates
    gates_rnn[:, :, i, :] = gates
```

학습) 순전파

- 입력 데이터를 시계열 순서로 GRU층을 통과시키면서 은닉 상태를 업데이트 한다.
- 각 시점에서의 은닉 상태와 게이트 값을 저장하여 역전파에서 활용할 수 있도록 한다.
- GRU층의 마지막 은닉 상태를 받아 출력층에서 최종 예측값을 계산한다.

실습

```
# 역전파 출력층
gru_layer.reset_sum_grad()
for i in reversed(range(n_time)):
    x = x_mb[:, i, :]
    y = y_rnn[:, i+1, :]
    y_prev = y_rnn[:, i, :]
    gates = gates_rnn[:, :, i, :]

    gru_layer.backward(x, y, y_prev, gates, grad_y)
    grad_y = gru_layer.grad_y_prev
```

학습) 역전파

- 예측값과 정답을 비교하며 출력층의 오차를 계산 과 GRU층으로 전달할 기울기 저장
- 이후 GRU층에선 시간을 거슬러 가며 역전파를 수행
- 각 시점의 은닉 상태와 게이트 값을 활용해 가중치의 기울기 계산, 기울기는 이전 시점의 은닉 상태로 계속 전달
- 이후 파라미터 갱신 (가중치 업데이트)

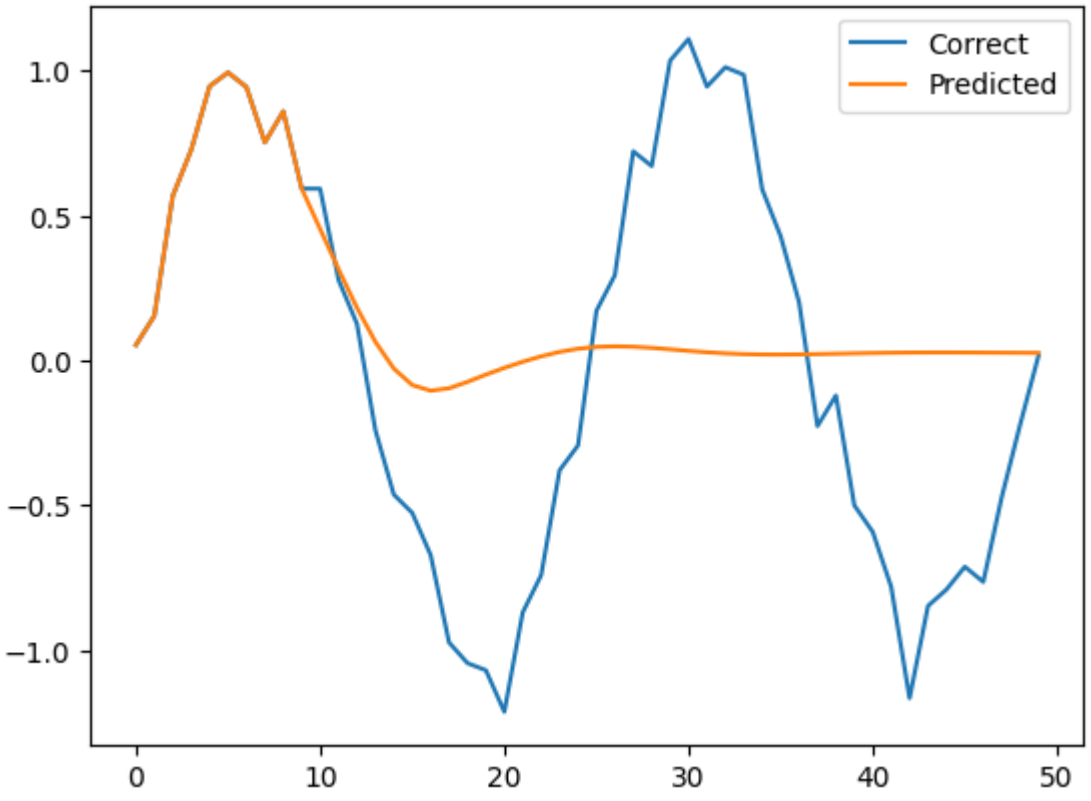
실습

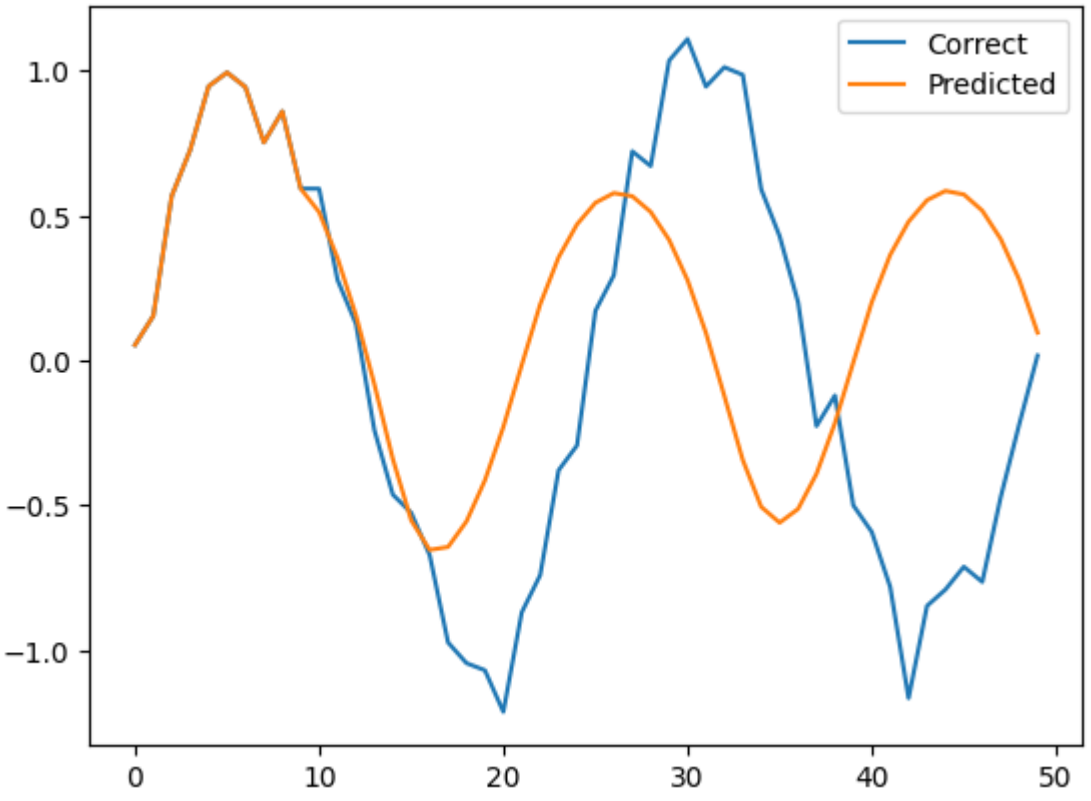
```python
# -- 예측--
def predict(x_mb):
    # 순전파 GRU층
    y_prev = np.zeros((len(x_mb), n_mid))
    for i in range(n_time):
        x = x_mb[:, i, :]
        gru_layer.forward(x, y_prev)
        y = gru_layer.y
        y_prev = y

    # 순전파 출력층
    output_layer.forward(y)
    return output_layer.y
```

예측

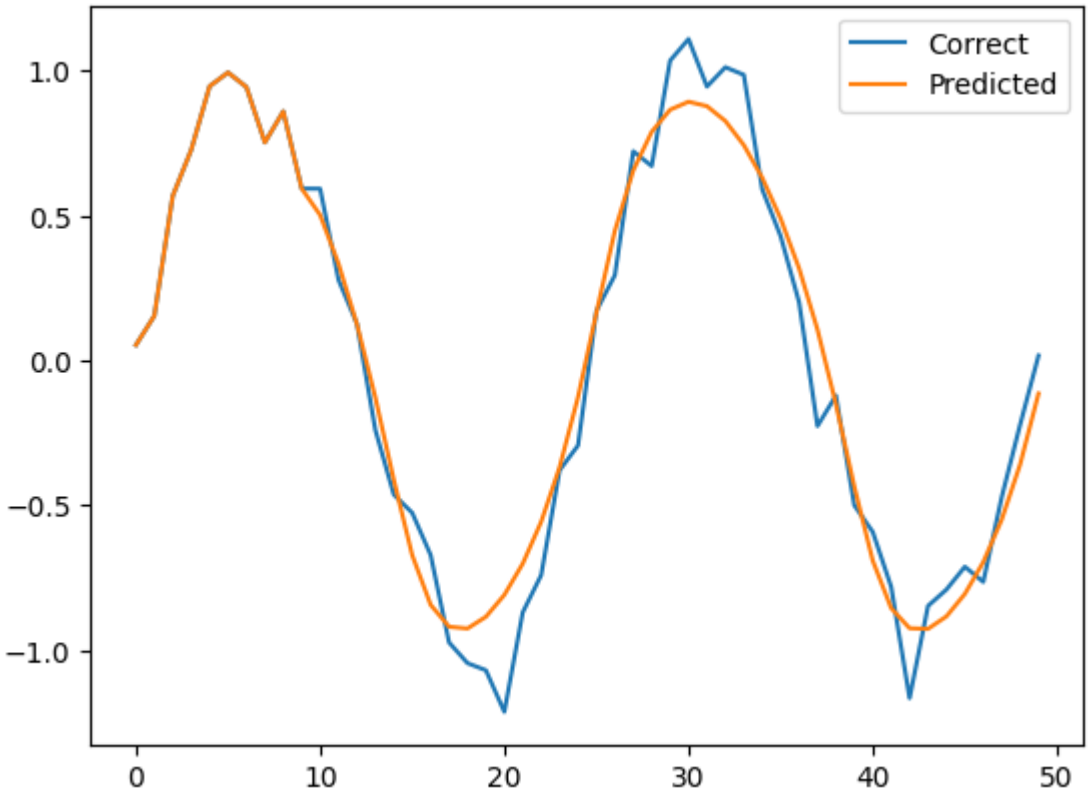
- 입력 데이터를 시간 순서대로 GRU층으로 통과시키면서 은닉 상태를 업데이트 한다.
- 학습 과정과 동일하게 업데이트 게이트와 리셋 게이트를 활용하여 새로운 은닉 상태를 계산 후 마지막 시점의 은닉 상태가 출력층으로 전달
- GRU층의 마지막 은닉 상태를 입력으로 사용하여 출력층에서 최종 예측값을 계산, 예측값을 반환하여 미래 값을 예측

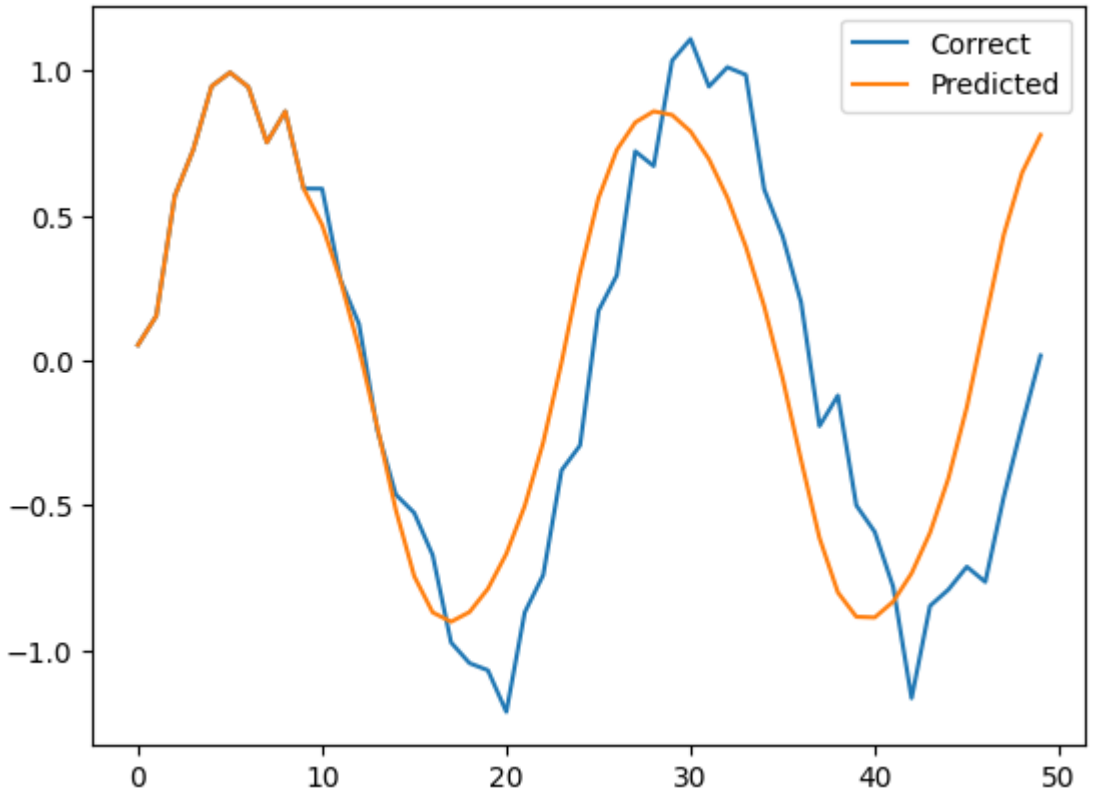
결과


Correct
Predicted
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50

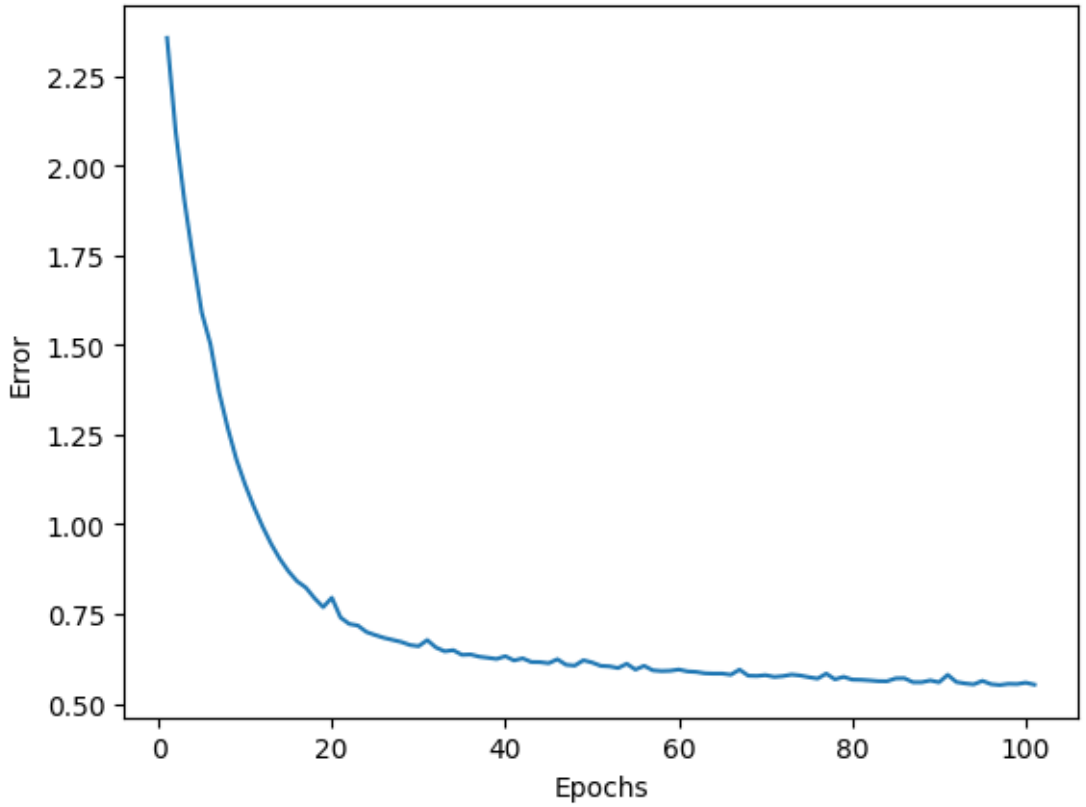

Correct
Predicted
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50

결과


Correct
Predicted
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50

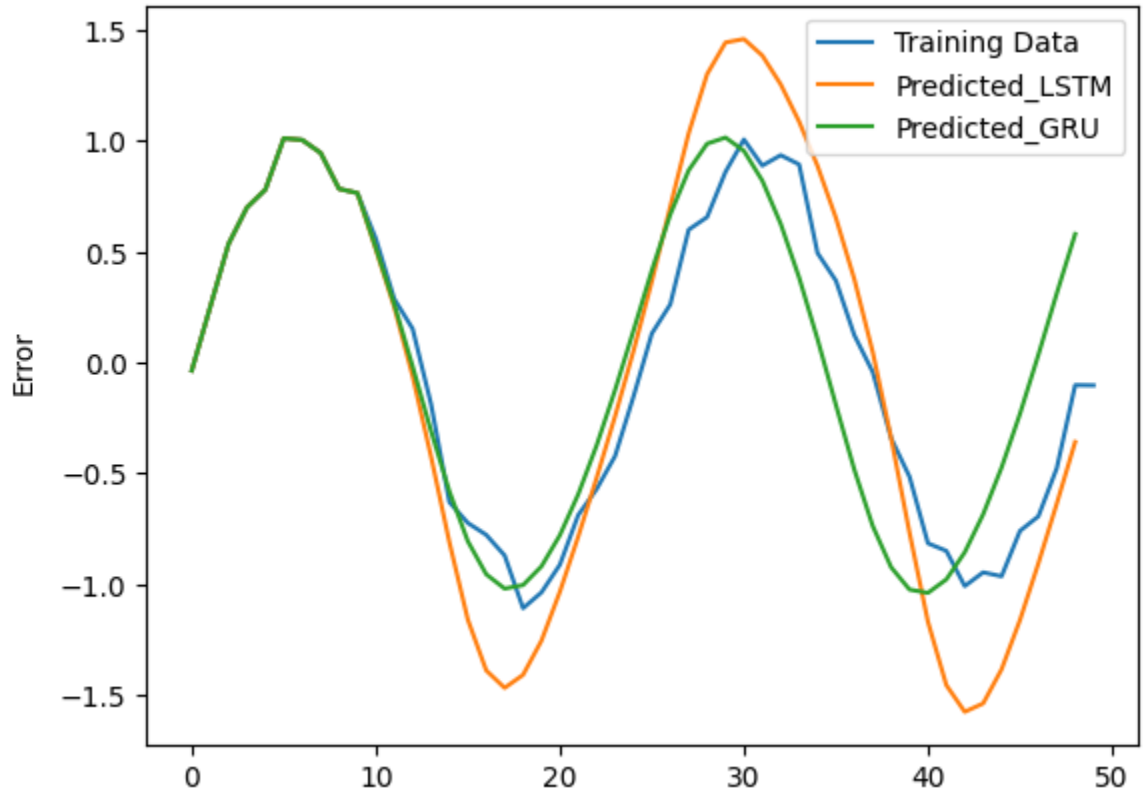

Correct
Predicted
1.0
0.5
0.0
−0.5
−1.0
0
10
20
30
40
50

실습



결과

- GRU로 예측해 생성된 sin곡선은 에포크가 늘수록 훈련 데이터에 가까운 곡선을 그린다.
- 그 오차가 지속적으로 줄어듦을 확인



LSTM과 비교

- GRU를 사용한 모델의 수렴이 LSTM보다 더 빠르고 원래의 데이터에 더 근접하다

결론

결론

- GRU는 단순하면서도 효과적인 RNN의 변형으로, 게이트 메커니즘을 활용해 장기 의존성 문제를 완화하고 연산량을 줄여 빠른 학습을 가능하게 한다.

장.단점

- LSTM과 유사한 성능으로 더 적은 계산량으로도 해결이 가능하지만, LSTM보다 미세한 제어가 어려울 수 있다.

활용 방안

- 시계열 데이터 처리와 예측에 강점을 가진 모델로, 시장 분석/수요예측/공정의 최적화/기술 로드맵(기술 발전 주기 예측, 기업 전략 수립) 등과 같은 부분에서 활용 될 수 있다.